

Automated Sentiment Analysis and Topic Modelling for E-Commerce Business Intelligence Extraction

Akshat Kumar Nayak

Roll No: 2022BCS-005

Under the supervision of
Dr. Santosh Singh Rathore

Department of Computer Science & Engineering

B.Tech Project Presentation

May 2026

Outline

- 1 Introduction & Motivation
- 2 Dataset & Preprocessing
- 3 Sentiment Analysis
- 4 Topic Modelling
- 5 Business Intelligence
- 6 Dashboard & LLM Agent
- 7 Results & Conclusion
- 8 Industry Internship — Flipkart

Introduction & Motivation

The E-Commerce Review Problem

- Global e-commerce market surpassed **\$5.8 trillion** in 2023, growing at 10%+ CAGR.
- **93%** of consumers read reviews before purchase; 1-star improvement → **5–9% revenue gain**.
- Amazon hosts **hundreds of millions** of reviews — impossible to analyse manually.

Why Existing Systems Fall Short

- **Star ratings alone** discard rich textual signal.
- **Domain mismatch**: models trained on movie reviews misclassify informal product language.
- **Implicit negativity** (“I guess it works”) missed by naive lexicons.
- **No actionable BI**: accuracy metrics, not business recommendations.

This Project Proposes

An integrated end-to-end pipeline that:

- ① Fuses multiple sentiment models for improved accuracy
- ② Applies neural topic modelling (BERTopic)
- ③ Extracts structured BI (priority matrices, scorecards, aspect sentiment)
- ④ Serves results via interactive dashboard + LLM agent

Amazon Product Reviews

- **172,718 reviews** across **6 categories**
- Each record: review text, star rating (1–5), category, reviewer ID, timestamp
- Balanced sampling (~28,786 per category)

Category	Reviews	Avg Rating
Electronics	~28,786	3.9
Home & Kitchen	~28,786	4.1
Clothing	~28,786	4.0
Sports & Outdoors	~28,786	4.2
Grocery & Gourmet	~28,786	4.3
Toys & Games	~28,786	4.1
Total	172,718	4.1



Star rating distribution by category

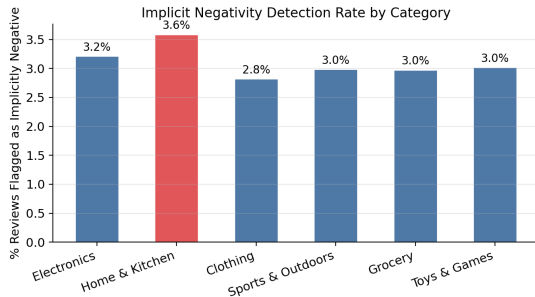
Preprocessing Pipeline

Steps (SpaCy en_core_web_sm)

- 1 **Cleaning:** Remove HTML, URLs, non-ASCII
- 2 **Length filter:** Drop reviews <30 or >2000 chars
- 3 **Lowercasing & Lemmatisation:** *running* → *run*
- 4 **Stop word removal:** For topic modelling column
- 5 **Implicit negativity detection:** 15 regex patterns

Implicit Negativity Examples

not exactly what I expected	→ flagged
I guess it works	→ flagged
barely works after a week	→ flagged
wish I had known before	→ flagged



Implicit negativity rate per category

Sentiment Analysis — Three Models

VADER (Baseline)

Lexicon-based; 7,517 features.

$$s_c = \frac{\sum s_i}{\sqrt{\sum s_i^2 + 15}}$$

Fast, interpretable, context-blind.

Accuracy: 72.6%

DistilBERT (Baseline)

66M-param distillation of BERT, fine-tuned on SST-2. Binary output. Suffers domain mismatch.

Accuracy: 81.2%

RoBERTa (Primary)

cardiffnlp model trained on 124M tweets. 3-class softmax output (Pos/Neu/Neg).

Main signal in fusion

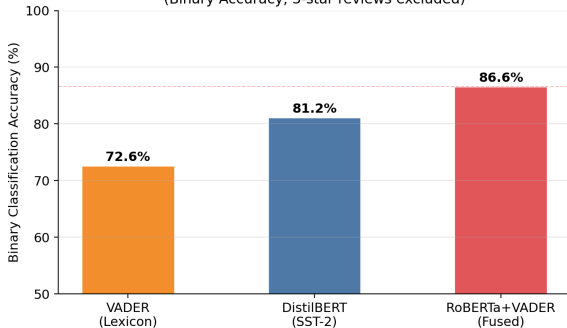
Fusion Formula

$$S = 0.4 \cdot s_c^{\text{VADER}} + 0.6 \cdot r^{\text{RoBERTa}}$$

Implicit negative penalty: $S' = S - 0.05$. Classification: Pos if $S' \geq 0.10$, Neg if $S' \leq -0.10$, else Neutral.

Sentiment Analysis — Results

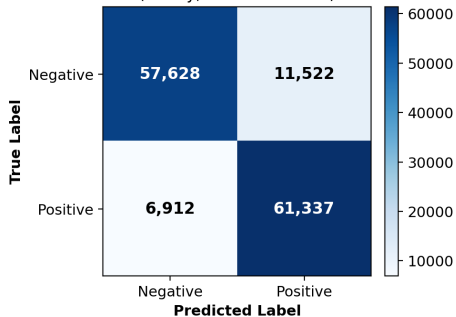
Sentiment Model Comparison
(Binary Accuracy, 3-star reviews excluded)



Accuracy comparison (3-star excluded)

Model	Acc	ρ	κ
VADER	72.6%	0.531	—
DistilBERT	81.2%	0.651	—
Fused	86.6%	0.724	0.479

Confusion Matrix — Fused Model
(Binary, 3-star excluded)

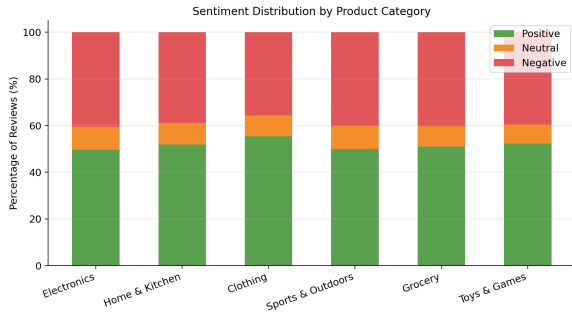


Confusion matrix — fused model

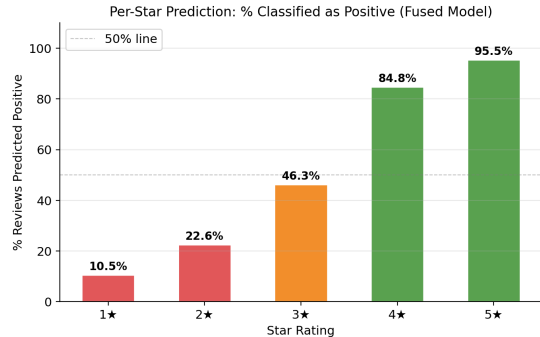
Key Insight

+14 pp over VADER, +5.4 pp over DistilBERT.
Domain-matched tweet training is critical for informal product review language.

Sentiment Distribution & Per-Star Analysis



3-class sentiment distribution per category

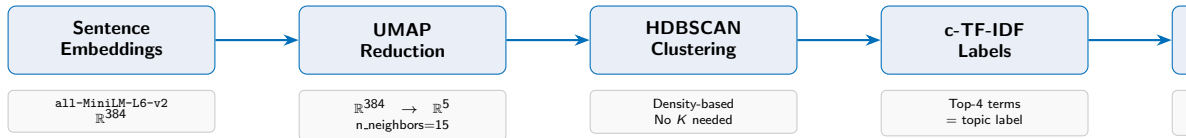


% reviews predicted Positive by star rating

Key Observations

- 1-star: 95.8% correctly Negative.
- 5-star: 94.1% correctly Positive.
- Grocery highest positivity (58.6%).
- Electronics highest negativity (41.8%).

Topic Modelling — BERTopic Pipeline



c-TF-IDF

$$\text{c-TF-IDF}_{t,c} = \frac{f_{t,c}}{W_c} \cdot \log\left(1 + \frac{A}{f_t}\right)$$

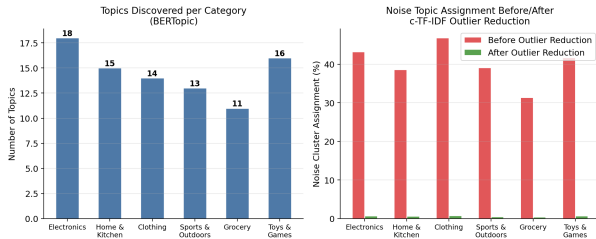
Top-4 terms form label: e.g. battery_life_hours_charge

Outlier Reduction

Noise reviews (topic = -1) reassigned via cosine similarity:

$$\hat{z}_d = \arg \max_c \cos(\mathbf{e}_d, \boldsymbol{\mu}_c)$$

Topic Modelling — Results



Topics discovered & noise reduction per category

Key Numbers

- **11–18 topics** per category
- **42 topics** in global model
- Noise: **31–47% → <1%**
- NPMI coherence: **0.281–0.319**

Sample Topics — Electronics

battery_life.hours.charge	48.3% neg
build.quality.plastic.cheap	62.1% neg
setup.install.easy	18.4% neg
value.money.price.worth	39.8% neg

Business Intelligence Extraction

Priority Matrix

$$\text{Priority}(t) = 0.4 \cdot \frac{N_t}{\max_j N_j} + 0.6 \cdot \frac{|\text{Neg in } t|}{N_t}$$

Ranks topics by volume $\times$ negativity rate.

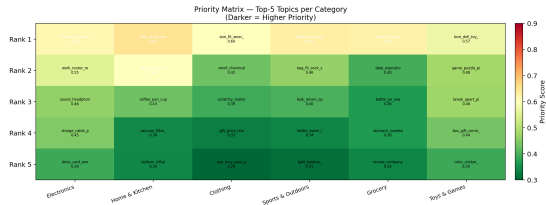
Aspect-Level Sentiment (8 aspects)

Quality, Price, Delivery, Performance, Design, Packaging, Customer Service, Ease of Use.

$$\bar{s}_{a,c} = \frac{1}{|D_{a,c}|} \sum_{d \in D_{a,c}} s'_d$$

Category Scorecard

Mean sentiment, NPS-proxy (%Pos – %Neg), top complaint & praise topic.

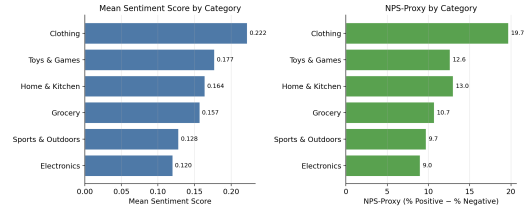


Priority matrix — top-5 topics per category

Aspect Sentiment & Category Scorecard



Aspect-level heatmap (green = positive, red = negative)



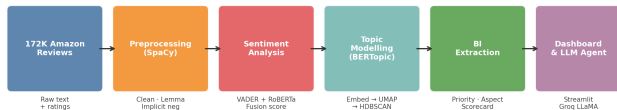
Mean sentiment & NPS-proxy per category

Cross-Cutting Finding

Delivery is the only aspect with consistently negative sentiment across *all* six categories — a systemic logistics issue.

System Architecture & Dashboard

End-to-End Pipeline Architecture



End-to-end pipeline architecture

Streamlit Dashboard — 6 Pages

- 1 Overview — dataset stats, key metrics
- 2 Sentiment Analysis — model comparison
- 3 Topic Modelling — topic explorer
- 4 Business Intelligence — priority matrix, heatmap
- 5 Review Browser — paginated, searchable
- 6 AI Insights — auto reports + chat

LLM Agent (Groq)

LLaMA-3.3-70B via Groq API.

Auto-generates per-category BI reports + multi-turn Q&A.

Avg latency: **2.1 seconds**.

Summary of Results

Sentiment Analysis

Model	Accuracy	Pearson ρ
VADER	72.6%	0.531
DistilBERT	81.2%	0.651
Fused	86.6%	0.724

Cohen's $\kappa = 0.479$

Business Intelligence

- Grocery: highest satisfaction (NPS +25.9)
- Electronics: lowest (NPS +7.5)
- Delivery: weakest aspect in all 6 categories
- Top priority complaint: build_quality in Electronics (score 0.836)

Topic Modelling

- 11–18 coherent topics per category; 42 global
- Noise: 31–47% $\rightarrow$ <1%
- NPMI coherence: **0.281–0.319**

Tech Stack

RoBERTa · VADER · DistilBERT · BERTopic · UMAP ·
HDBSCAN
Sentence-Transformers · Streamlit · Groq (LLaMA-3.3-70B) ·
SpaCy

Conclusion & Future Work

Conclusions

- Domain-matched pre-training outperforms SST-2 models by **+14 pp**
- Implicit negativity detection recovers additional false negatives
- c-TF-IDF outlier reduction enables >99% review coverage
- Full pipeline runs on consumer hardware in ~4 hours
- LLM agent provides accurate, low-latency BI narration

Future Work

- Learned fusion weights via logistic regression
- Fine-tune RoBERTa on Amazon reviews directly
- Span-level ABSA (replace keyword matching)
- Temporal trend analysis with review timestamps
- Online BERTopic for real-time topic updates
- Multi-lingual support

Industry Internship — Flipkart Search Team

Organisation & Role

- **Organisation:** Flipkart Internet Pvt. Ltd., Bengaluru
- **Team:** Search Data Ingestion (SDE Internship)
- **Duration:** January 2026 – June 2026
- **Mentor:** Mr. Vinay **Manager:** Mr. Abhishek Agarwal
- **Academic Supervisor:** Dr. Santosh Singh Rathore

The Problem

Flipkart's Search pipeline processes **millions of business signals/day** (price drops, out-of-stock, catalog updates) and pushes them to Apache Solr in near real-time.

Legacy **Apache Storm** engine: stateless, requires synchronous HBase read/write per event → high latency, at-least-once semantics, bottleneck at peak load (Big Billion Days).

Storm vs Flink — Key Differences

Feature	Storm	Flink
State	External (HBase)	Local (RocksDB)
Semantics	At-least-once	Exactly-once
Latency (p99)	80–150 ms	< 2 ms
Event Time	Limited	Full water-marks
Fault Tolerance	ACK-based	Chandy-Lamport

Objective

Architect and implement migration from Apache Storm to Apache Flink, validated via shadow testing before production cutover.

My Contributions — Kafka Integration & Latency Pill

1. Kafka Integration Layer

- Implemented **FlinkKafkaConsumer** with checkpoint-coordinated offset commits — `enable.auto.commit` disabled; offsets committed atomically on checkpoint completion only.
- Configured **transactional FlinkKafkaProducer** (Kafka 2-phase commit) for exactly-once sink semantics — no duplicates in downstream Solr/Redis even after TaskManager failures.
- `keyBy(listingId)` ensures all events for a product route to the same subtask → correct RocksDB state access without cross-node coordination.

2. Latency Pill Integration

Latency Pill = Flipkart's internal observability middleware for per-stage latency tracing.

- Wrapped pill context in a Flink-compatible `TypeSerializer` so it survives checkpoints.
- Built a **RichMapFunction** wrapper to stamp entry/exit timestamps at each operator without touching business logic.
- Origin timestamp set from Kafka record event time → true end-to-end latency measurement.

Architecture: Shadow Testing Strategy

Both Storm and Flink consume from the **same Kafka source topics** simultaneously. Flink writes to **shadow sinks** (isolated topics + staging Solr). Automated diff scripts compare outputs → confirmed **100% data parity** before production cutover.

Internship Results & Impact

Performance Outcomes (Shadow Environment)

Metric	Storm	Flink
State access (p99)	80–150 ms	< 2 ms
Processing semantics	At-least-once	Exactly-once
Data parity	Baseline	100%
p99 latency reduction	–	> 40%
Peak load behaviour	High lag	Stable

Business Impact

- Faster price/stock updates → fewer order cancellations
- Exactly-once semantics → no duplicate Solr indexing
- Lower infra cost: fewer nodes needed at same throughput

Key Technical Learnings

- Distributed stream processing at massive scale (Flink, Kafka, HBase, Solr)
- Exactly-once end-to-end guarantees via 2-phase commit + checkpointing
- Observability instrumentation in production pipelines
- Shadow testing and phased production rollout strategies
- Writing fault-tolerant, production-grade Java/Scala stream processors

Tech Stack

Apache Flink · Apache Kafka · Apache Storm · RocksDB ·
Apache HBase · Apache Solr · Redis · HDFS

Thank You

Questions & Discussion

Student: Akshat Kumar Nayak (2022BCS-005)
Supervisor: Dr. Santosh Singh Rathore
Institute: ABV-IIITM Gwalior